

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
“КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО”

Факультет програмних систем та прикладної математики
Кафедра програмного забезпечення комп’ютерних систем




**Лабораторна робота № 3**
з дисципліни “Розроблення вебзастосунків”
тема «Додаткові можливості серверних вебзастосунків»

Виконав
студент III курсу
групи КП-42
Жижка Богдан

Київ 2026

**Мета роботи:** Отримати навички роботи з додатковими можливостями серверних вебзастосунків, а саме конфігурацією, кешуванням та логуванням.

**Завдання:**

Додати до серверного вебзастосунку, реалізованого в ході виконання першої та другої робіт комп'ютерного практикуму, наступні функціональні можливості:

1. Логування (наприклад, під час виклику методів репозиторіїв, реалізованих в ході виконання другої роботи комп'ютерного практикуму).
2. Кешування для тих операцій API, де це є доцільним.
3. Роботу з конфігурацією, яка також повинна впливати на поведінку REST API для користувача.

## ХІД РОБОТИ

**1. Постановка задачі** Для виконання комп'ютерного практикуму використано предметну галузь з першої роботи — «Система управління таксопарком». Система призначена для обліку автомобілів таксопарку, а також фіксації водіїв, що за ними закріплені. У третій роботі до застосунку додано логування, кешування та роботу з конфігурацією. Усі необхідні об'єкти отримуються через IoC-контейнер.

Сутності мають між собою зв'язок «один-до-багатьох»: один автомобіль може мати багато водіїв, що працюють позмінно, а кожен водій закріплений за одним автомобілем. При видаленні автомобіля його водії видаляються каскадно.

**Автомобіль (Car)** — сутність, що репрезентує транспортний засіб у системі. Містить наступні властивості:

- Id (ціле число) — унікальний ідентифікатор автомобіля, генерується базою даних.
- Brand (рядок) — марка автомобіля.
- Model (рядок) — модель автомобіля.
- EngineVolume (дійсне число) — об'єм двигуна.
- Year (ціле число) — рік випуску транспортного засобу.

- Age (ціле число) — вік автомобіля, обчислюється під час формування відповіді та в БД не зберігається.

**Водій (Driver)** — сутність, що зберігає дані про працівника таксопарку. Містить наступні властивості:

- Id (ціле число) — унікальний ідентифікатор водія, генерується базою даних.
- FullName (рядок) — повне ім’я водія.
- Rating (дійсне число) — рейтинг водія в системі оцінювання.
- IsBusy (логічне значення) — поточний статус водія (вільний або виконує замовлення).
- CarId (ціле число) — зовнішній ключ на автомобіль, за яким закріплений водій.

**Налаштування конфігурації.** Налаштування зберігаються у файлі appsettings.json і зчитуються за допомогою шаблону Options. Зміни у файлі застосовуються без перезапуску програми, а некоректні значення перевіряються під час запуску.

- MaxDriversPerCar (3) — максимальна кількість водіїв на один автомобіль. Якщо ліміт вичерпано, API повертає код 409 Conflict.
- MaxCarAgeYears (15) — максимальний вік автомобіля. Автомобіль, старший за цей вік, не можна додати — API повертає код 400 Bad Request.
- AllowDeleteAll (false) — дозвіл на видалення всіх автомобілів або водіїв. Якщо false, API повертає код 403 Forbidden.
- Caching:Enabled (true) — вмикає або вимикає кешування.
- Caching:DurationSeconds (60) — час зберігання даних у кеші в секундах.

**Логування.** Для логування використано бібліотеку Serilog. Логи записуються в консоль, текстовий файл та файл у форматі JSON. Логуються звернення до бази даних у репозиторіях (з часом виконання), операції сервісів, робота кешу, помилки валідації та кожен HTTP-запит. Кожен запис має ідентифікатор події (EventId).

**Кешування.** Кешування реалізовано в пам’яті сервера за допомогою IMemoryCache. Кешуються GET-запити на отримання автомобілів і водіїв.

Після створення, оновлення або видалення даних відповідний кеш очищується, щоб користувач отримував актуальні дані.

## Код розробленого програмного забезпечення

## Caching

```
namespace Taxi.Caching;


public static class CacheKeys
{
    public const string AllCars = "cars:all";
    public const string AllDrivers = "drivers:all";


    public static string Car(int id) => $"cars:{id}";
    public static string CarDrivers(int carId) => $"cars:{carId}:drivers";
    public static string Driver(int id) => $"drivers:{id}";
}
```

```
using Microsoft.Extensions.Caching.Memory;
using Microsoft.Extensions.Options;
using Microsoft.Extensions.Primitives;
using Taxi.Logging;
using Taxi.Options;


namespace Taxi.Caching;


public class CacheService : ICacheService
{
    private readonly IMemoryCache cache;
    private readonly IOptionsMonitor<CachingOptions> options;
    private readonly ILogger<CacheService> logger;
    private CancellationTokenSource resetTokenSource = new();
    private readonly object resetLock = new();


    public CacheService(IMemoryCache cache, IOptionsMonitor<CachingOptions>
options, ILogger<CacheService> logger)
    {
```

```
        this.cache = cache;
        this.options = options;
        this.logger = logger;
    }

    public async Task<T> GetOrCreateAsync<T>(string key, Func<Task<T>> factory)
    {
        var settings = options.CurrentValue;

        if (!settings.Enabled)
        {
            logger.LogInformation(LogEvents.CacheDisabled, "[Кеш] Вимкнено в конфігурації — {Key} завантажується з БД", key);
            return await factory();
        }

        if (cache.TryGetValue(key, out T? cachedValue) && cachedValue is not null)
        {
            logger.LogInformation(LogEvents.CacheHit, "[Кеш] Влучання: {Key}", key);
            return cachedValue;
        }

        logger.LogInformation(LogEvents.CacheMiss, "[Кеш] Промах: {Key} — завантаження з БД", key);
        var value = await factory();

        var entryOptions = new MemoryCacheEntryOptions()
            .SetAbsoluteExpiration(TimeSpan.FromSeconds(settings.DurationSeconds))
            .AddExpirationToken(new CancellationChangeToken(resetTokenSource.Token));

        cache.Set(key, value, entryOptions);
```

```
        logger.LogInformation(LogEvents.CacheStored, "[Кеш] Збережено: {Key} на
{Seconds} с", key, settings.DurationSeconds);


        return value;

    }


    public void Remove(params string[] keys)

    {

        var distinctKeys = keys.Distinct().ToArray();

        foreach (var key in distinctKeys)

        {

            cache.Remove(key);

        }


        logger.LogInformation(LogEvents.CacheRemoved, "[Кеш] Скинуто: {Keys}",
string.Join(", ", distinctKeys));

    }


    public void Clear()

    {

        CancellationTokenSource oldTokenSource;

        lock (resetLock)

        {

            oldTokenSource = resetTokenSource;

            resetTokenSource = new CancellationTokenSource();

        }


        oldTokenSource.Cancel();

        logger.LogInformation(LogEvents.CacheCleared, "[Кеш] Повністю очищено");

    }

}
```

```
namespace Taxi.Caching;
```

```
public interface ICacheService

{

   Task<T> GetOrCreateAsync<T>(string key, Func<Task<T>> factory);

   void Remove(params string[] keys);

   void Clear();

}
```

## Controllers

```
using Microsoft.AspNetCore.Mvc;

using Taxi.DTOs;

using Taxi.Services;


namespace Taxi.Controllers;


[ApiController]

[Route("api/[controller]")]

[Produces("application/json")]

public class CarsController : ControllerBase

{

   private readonly ICarService carService;


   public CarsController(ICarService carService)

   {

       this.carService = carService;

   }


   [HttpGet]

   [ProducesResponseType(typeof(List<CarResponseDto>),
StatusCodes.Status200OK)]

   public async Task<ActionResult<List<CarResponseDto>>> GetAll()

   {

       var cars = await carService.GetAllAsync();

       return Ok(cars);
```

```
    }


    [HttpGet("{id:int}")]
    [ProducesResponseType(typeof(CarResponseDto), StatusCodes.Status200OK)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    public async Task<ActionResult<CarResponseDto>> GetById(int id)
    {
        var car = await carService.GetByIdAsync(id);
        return Ok(car);
    }


    [HttpPost]
    [ProducesResponseType(typeof(CarResponseDto), StatusCodes.Status201Created)]
    [ProducesResponseType(StatusCodes.Status400BadRequest)]
    [ProducesResponseType(StatusCodes.Status409Conflict)]
    public async Task<ActionResult<CarResponseDto>> Create([FromBody]
CarCreateDto dto)
    {
        var created = await carService.CreateAsync(dto);
        return CreatedAtAction(nameof(GetById), new { id = created.Id },
created);
    }


    [HttpPut("{id:int}")]
    [ProducesResponseType(typeof(CarResponseDto), StatusCodes.Status200OK)]
    [ProducesResponseType(StatusCodes.Status400BadRequest)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    [ProducesResponseType(StatusCodes.Status409Conflict)]
    public async Task<ActionResult<CarResponseDto>> Update(int id, [FromBody]
CarUpdateDto dto)
    {
        var updated = await carService.UpdateAsync(id, dto);
        return Ok(updated);
    }
```

```
    [HttpDelete("{id:int}")]
    [ProducesResponseType(StatusCodes.Status204NoContent)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    public async Task<IActionResult> Delete(int id)
    {
        await carService.DeleteAsync(id);
        return NoContent();
    }

    [HttpDelete]
    [ProducesResponseType(StatusCodes.Status204NoContent)]
    public async Task<IActionResult> DeleteAll()
    {
        await carService.DeleteAllAsync();
        return NoContent();
    }

    [HttpGet("{id:int}/drivers")]
    [ProducesResponseType(typeof(List<DriverResponseDto>),
StatusCodes.Status200OK)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    public async Task<ActionResult<List<DriverResponseDto>>>
GetDriversByCarId(int id)
    {
        var drivers = await carService.GetDriversByCarIdAsync(id);
        return Ok(drivers);
    }
}
```

```
using Microsoft.AspNetCore.Mvc;
using Taxi.DTOs;
using Taxi.Services;
```

```
namespace Taxi.Controllers;


[ApiController]
[Route("api/[controller]")]
[Produces("application/json")]
public class DriversController : ControllerBase
{
    private readonly IDriverService driverService;


    public DriversController(IDriverService driverService)
    {
        this.driverService = driverService;
    }


    [HttpGet]
    [ProducesResponseType(typeof(List<DriverResponseDto>),
StatusCodes.Status200OK)]
    public async Task<ActionResult<List<DriverResponseDto>>> GetAll()
    {
        var drivers = await driverService.GetAllAsync();
        return Ok(drivers);
    }


    [HttpGet("{id:int}")]
    [ProducesResponseType(typeof(DriverResponseDto), StatusCodes.Status200OK)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    public async Task<ActionResult<DriverResponseDto>> GetById(int id)
    {
        var driver = await driverService.GetByIdAsync(id);
        return Ok(driver);
    }


    [HttpPost]
```

```
    [ProducesResponseType(typeof(DriverResponseDto),
StatusCodes.Status201Created)]
    [ProducesResponseType(StatusCodes.Status400BadRequest)]
    [ProducesResponseType(StatusCodes.Status409Conflict)]
    public async Task<ActionResult<DriverResponseDto>> Create([FromBody]
DriverCreateDto dto)
    {
        var created = await driverService.CreateAsync(dto);
        return CreatedAtAction(nameof(GetById), new { id = created.Id },
created);
    }

    [HttpPut("{id:int}")]
    [ProducesResponseType(typeof(DriverResponseDto), StatusCodes.Status200OK)]
    [ProducesResponseType(StatusCodes.Status400BadRequest)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    [ProducesResponseType(StatusCodes.Status409Conflict)]
    public async Task<ActionResult<DriverResponseDto>> Update(int id, [FromBody]
DriverUpdateDto dto)
    {
        var updated = await driverService.UpdateAsync(id, dto);
        return Ok(updated);
    }

    [HttpDelete("{id:int}")]
    [ProducesResponseType(StatusCodes.Status204NoContent)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    public async Task<IActionResult> Delete(int id)
    {
        await driverService.DeleteAsync(id);
        return NoContent();
    }

    [HttpDelete]
```

```
    [ProducesResponseType(StatusCodes.Status204NoContent)]
    public async Task<IActionResult> DeleteAll()
    {
        await driverService.DeleteAllAsync();
        return NoContent();
    }
}
```

## Data

```
using Microsoft.EntityFrameworkCore;
using Taxi.Models;

namespace Taxi.Data;

public class TaxiDbContext : DbContext
{
    public TaxiDbContext(DbContextOptions<TaxiDbContext> options) :
base(options) {  }
    public DbSet<Car> Cars => Set<Car>();
    public DbSet<Driver> Drivers => Set<Driver>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        modelBuilder.Entity<Car>(entity =>
        {
            entity.ToTable("Cars");
            entity.HasKey(c => c.Id);
            entity.Property(c => c.Brand).IsRequired().HasMaxLength(50);
            entity.Property(c => c.Model).IsRequired().HasMaxLength(50);
            entity.Property(c => c.EngineVolume).IsRequired();
            entity.Property(c => c.Year).IsRequired();
```

```
        });


        modelBuilder.Entity<Driver>(entity =>
        {
            entity.ToTable("Drivers", t =>
t.HasCheckConstraint("CK_Drivers_Rating", "\"Rating\" >= 0 AND \"Rating\" <=
5"));
            entity.HasKey(d => d.Id);
            entity.Property(d => d.FullName).IsRequired().HasMaxLength(100);
            entity.Property(d => d.Rating).IsRequired();
            entity.Property(d => d.IsBusy).IsRequired();
            entity.HasOne(d => d.Car).WithMany(c => c.Drivers).HasForeignKey(d
=> d.CarId).OnDelete(DeleteBehavior.Cascade);
        });
    }
}
```

## DTOs

```
namespace Taxi.DTOs;


public abstract class CarBaseDto
{
    public string Brand { get; set; } = string.Empty;
    public string Model { get; set; } = string.Empty;
    public double EngineVolume { get; set; }
    public int Year { get; set; }
}
```

```
namespace Taxi.DTOs;


public class CarCreateDto : CarBaseDto
{
}
```

```
namespace Taxi.DTOs;


public class CarResponseDto
{
    public int Id { get; set; }
    public string Brand { get; set; } = string.Empty;
    public string Model { get; set; } = string.Empty;
    public double EngineVolume { get; set; }
    public int Year { get; set; }
    public int Age { get; set; }
}
```

```
namespace Taxi.DTOs;


public class CarUpdateDto : CarBaseDto
{
}
```

```
namespace Taxi.DTOs;


public abstract class DriverBaseDto
{
    public string FullName { get; set; } = string.Empty;
    public double Rating { get; set; }
    public bool IsBusy { get; set; }
    public int CarId { get; set; }
}
```

```
namespace Taxi.DTOs;


public class DriverCreateDto : DriverBaseDto
{
}
```

```
namespace Taxi.DTOs;


public class DriverResponseDto
{
    public int Id { get; set; }
    public string FullName { get; set; } = string.Empty;
    public double Rating { get; set; }
    public bool IsBusy { get; set; }
    public int CarId { get; set; }
}
```

```
namespace Taxi.DTOs;


public class DriverUpdateDto : DriverBaseDto
{
}
```

## Exceptions

```
namespace Taxi.Exceptions;


public class BadRequestException : Exception
{
    public BadRequestException(string message) : base(message)
    {
    }
}
```

```
namespace Taxi.Exceptions;


public class ConflictException : Exception
{
    public ConflictException(string message) : base(message)
```

```
    {

    }
}
```

```
namespace Taxi.Exceptions;


public class ForbiddenException : Exception
{
    public ForbiddenException(string message) : base(message)
    {

    }
}
```

```
namespace Taxi.Exceptions;


public class NotFoundException : Exception
{
    public NotFoundException(string message) : base(message)
    {

    }
}
```

## Logging

```
namespace Taxi.Logging;


public static class LogEvents
{
    public const int CarsGetAll = 1001;

    public const int CarGetById = 1002;

    public const int CarCreated = 1003;

    public const int CarUpdated = 1004;

    public const int CarDeleted = 1005;

    public const int CarsDeletedAll = 1006;
```

```
    public const int CarDriversGet = 1007;
    public const int CarNotFound = 1404;

    public const int DriversGetAll = 2001;
    public const int DriverGetById = 2002;
    public const int DriverCreated = 2003;
    public const int DriverUpdated = 2004;
    public const int DriverDeleted = 2005;
    public const int DriversDeletedAll = 2006;
    public const int DriverCarMissing = 2400;
    public const int DriverNotFound = 2404;
    public const int DriverLimitReached = 2409;

    public const int DbRead = 3001;
    public const int DbWrite = 3002;
    public const int DbDelete = 3003;

    public const int CacheHit = 4001;
    public const int CacheMiss = 4002;
    public const int CacheStored = 4003;
    public const int CacheRemoved = 4004;
    public const int CacheCleared = 4005;
    public const int CacheDisabled = 4006;

    public const int ValidationFailed = 5400;
    public const int Forbidden = 5403;
    public const int UnhandledError = 5500;
}
```

## Mappings

```
using AutoMapper;
using Taxi.DTOs;
using Taxi.Models;
```

```
using Taxi.Validators;


namespace Taxi.Mappings;


public class MappingProfile : Profile

{

   public MappingProfile()

   {

       CreateMap<Car, CarResponseDto>()

           .ForMember(dest => dest.Age, opt => opt.MapFrom(src =>
DateTime.UtcNow.Year - src.Year));



       CreateMap<CarBaseDto, Car>()

           .ForMember(dest => dest.Id, opt => opt.Ignore())

           .ForMember(dest => dest.Drivers, opt => opt.Ignore())

           .ForMember(dest => dest.Brand, opt => opt.MapFrom(src =>
src.Brand.Trim()))

           .ForMember(dest => dest.Model, opt => opt.MapFrom(src =>
src.Model.Trim()));



       CreateMap<CarCreateDto, Car>().IncludeBase<CarBaseDto, Car>();

       CreateMap<CarUpdateDto, Car>().IncludeBase<CarBaseDto, Car>();



       CreateMap<Driver, DriverResponseDto>();



       CreateMap<DriverBaseDto, Driver>()

           .ForMember(dest => dest.Id, opt => opt.Ignore())

           .ForMember(dest => dest.Car, opt => opt.Ignore())

           .ForMember(dest => dest.FullName, opt => opt.MapFrom(src =>
src.FullName.Trim()));



       CreateMap<DriverCreateDto, Driver>().IncludeBase<DriverBaseDto,
Driver>();

       CreateMap<DriverUpdateDto, Driver>().IncludeBase<DriverBaseDto,
Driver>();
```

```
    }
}
```

## Middleware

```
using System.Text.Encodings.Web;
using System.Text.Json;
using System.Text.Json.Serialization;
using System.Text.Unicode;
using FluentValidation;
using Taxi.Exceptions;
using Taxi.Logging;

namespace Taxi.Middleware;

public class ExceptionHandlingMiddleware
{
    private static readonly JsonSerializerOptions JsonOptions = new()
    {
        PropertyNamingPolicy = JsonNamingPolicy.CamelCase,
        DefaultIgnoreCondition = JsonIgnoreCondition.WhenWritingNull,
        Encoder = JavaScriptEncoder.Create(UnicodeRanges.All)
    };

    private readonly RequestDelegate next;
    private readonly ILogger<ExceptionHandlingMiddleware> logger;

    public ExceptionHandlingMiddleware(RequestDelegate next,
ILogger<ExceptionHandlingMiddleware> logger)
    {
        this.next = next;
        this.logger = logger;
    }
```

```
    public async Task InvokeAsync(HttpContext context)
    {
        try
        {
            await next(context);
        }
        catch (Exception ex)
        {
            if (context.Response.HasStarted)
            {
                logger.LogError(LogEvents.UnhandledError, ex, "Помилка після початку відправлення відповіді.");
                throw;
            }

            await HandleExceptionAsync(context, ex);
        }
    }

    private async Task HandleExceptionAsync(HttpContext context, Exception exception)
    {
        int statusCode;
        string message;
        Dictionary<string, string[]>? errors = null;

        switch (exception)
        {
            case ValidationException validationException:
                statusCode = StatusCodes.Status400BadRequest;
                message = "Помилка валідації вхідних даних.";
                errors = validationException.Errors.GroupBy(e => e.PropertyName).ToDictionary(g => g.Key, g => g.Select(e => e.ErrorMessage).Distinct().ToArray());
```

```
                logger.LogWarning(LogEvents.ValidationFailed, "[Валідація]
{Method} {Path}: {Errors}", context.Request.Method, context.Request.Path,
string.Join("; ", validationException.Errors.Select(e => $"{e.PropertyName} —
{e.ErrorMessage}")));
                break;

            case BadRequestException:
                statusCode = StatusCodes.Status400BadRequest;
                message = exception.Message;
                break;

            case ForbiddenException:
                statusCode = StatusCodes.Status403Forbidden;
                message = exception.Message;
                break;

            case NotFoundException:
                statusCode = StatusCodes.Status404NotFound;
                message = exception.Message;
                break;

            case ConflictException:
                statusCode = StatusCodes.Status409Conflict;
                message = exception.Message;
                break;

            default:
                statusCode = StatusCodes.Status500InternalServerError;
                message = "Внутрішня помилка сервера.";
                logger.LogError(LogEvents.UnhandledError, exception,
                    "Необроблений виняток під час виконання запиту {Method}
{Path}",
                    context.Request.Method, context.Request.Path);
                break;
```

```
        }


        context.Response.Clear();

        context.Response.StatusCode = statusCode;

        context.Response.ContentType = "application/json; charset=utf-8";


        var payload = new ErrorResponse(statusCode, message, errors);

        await context.Response.WriteAsync(JsonSerializer.Serialize(payload,
JsonOptions));

    }


    private sealed record ErrorResponse(int Status, string Error,
Dictionary<string, string[]>? Errors);

}
```

## Models

```
namespace Taxi.Models;


public class Car

{

    public int Id { get; set; }

    public string Brand { get; set; } = string.Empty;

    public string Model { get; set; } = string.Empty;

    public double EngineVolume { get; set; }

    public int Year { get; set; }

    public ICollection<Driver> Drivers { get; set; } = new List<Driver>();

}
```

```
namespace Taxi.Models;


public class Driver

{

    public int Id { get; set; }
```

```
    public string FullName { get; set; } = string.Empty;

    public double Rating { get; set; }

    public bool IsBusy { get; set; }

    public int CarId { get; set; }

    public Car? Car { get; set; }
}
```

## Options

```
using System.ComponentModel.DataAnnotations;


namespace Taxi.Options;


public class CachingOptions
{
    public const string SectionName = "Caching";


    public bool Enabled { get; set; } = true;


    [Range(1, 3600, ErrorMessage = "Caching:DurationSeconds має бути від 1 до
3600.")]
    public int DurationSeconds { get; set; } = 60;
}
```

```
using System.ComponentModel.DataAnnotations;


namespace Taxi.Options;


public class TaxiFleetOptions
{
    public const string SectionName = "TaxiFleet";


    [Range(1, 10, ErrorMessage = "TaxiFleet:MaxDriversPerCar має бути від 1 до
10.")]
```

```
    public int MaxDriversPerCar { get; set; } = 3;


    [Range(1, 50, ErrorMessage = "TaxiFleet:MaxCarAgeYears має бути від 1 до
50.")]
    public int MaxCarAgeYears { get; set; } = 15;


    public bool AllowDeleteAll { get; set; }
}
```

## Repositories

```
using System.Diagnostics;
using Microsoft.EntityFrameworkCore;
using Taxi.Data;
using Taxi.Logging;
using Taxi.Models;


namespace Taxi.Repositories;


public class CarRepository : ICarRepository
{
    private readonly TaxiDbContext context;
    private readonly ILogger<CarRepository> logger;


    public CarRepository(TaxiDbContext context, ILogger<CarRepository> logger)
    {
        this.context = context;
        this.logger = logger;
    }


    public async Task<List<Car>> GetAllAsync()
    {
        var stopwatch = Stopwatch.StartNew();
```

```
        var cars = await context.Cars.AsNoTracking().OrderBy(c =>
c.Id).ToListAsync();

        logger.LogDebug(LogEvents.DbRead, "[БД] Отримано автомобілів: {Count}
({Elapsed} мс)", cars.Count, stopwatch.ElapsedMilliseconds);

        return cars;

    }


    public async Task<Car?> GetByIdAsync(int id)

    {

        var stopwatch = Stopwatch.StartNew();

        var car = await context.Cars.FirstOrDefaultAsync(c => c.Id == id);

        logger.LogDebug(LogEvents.DbRead, "[БД] Пошук автомобіля ID {Id}:
{Result} ({Elapsed} мс)", id, car is null ? "не знайдено" : "знайдено",
stopwatch.ElapsedMilliseconds);

        return car;

    }


    public async Task<bool> ExistsAsync(int id)

    {

        var stopwatch = Stopwatch.StartNew();

        var exists = await context.Cars.AnyAsync(c => c.Id == id);

        logger.LogDebug(LogEvents.DbRead, "[БД] Перевірка існування автомобіля
ID {Id}: {Result} ({Elapsed} мс)", id, exists ? "існує" : "не існує",
stopwatch.ElapsedMilliseconds);

        return exists;

    }


    public async Task AddAsync(Car car)

    {

        var stopwatch = Stopwatch.StartNew();

        await context.Cars.AddAsync(car);

        await context.SaveChangesAsync();

        logger.LogDebug(LogEvents.DbWrite, "[БД] Додано автомобіль ID {Id}
({Elapsed} мс)", car.Id, stopwatch.ElapsedMilliseconds);

    }
```

```
    public async Task UpdateAsync(Car car)
    {
        var stopwatch = Stopwatch.StartNew();
        context.Cars.Update(car);
        await context.SaveChangesAsync();
        logger.LogDebug(LogEvents.DbWrite, "[БД] Оновлено автомобіль ID {Id} ({Elapsed} мс)", car.Id, stopwatch.ElapsedMilliseconds);
    }

    public async Task DeleteAsync(Car car)
    {
        var stopwatch = Stopwatch.StartNew();
        context.Cars.Remove(car);
        await context.SaveChangesAsync();
        logger.LogDebug(LogEvents.DbDelete, "[БД] Видалено автомобіль ID {Id} ({Elapsed} мс)", car.Id, stopwatch.ElapsedMilliseconds);
    }

    public async Task<int> DeleteAllAsync()
    {
        var stopwatch = Stopwatch.StartNew();
        var deleted = await context.Cars.ExecuteDeleteAsync();
        logger.LogDebug(LogEvents.DbDelete, "[БД] Видалено всі автомобілі: {Count} ({Elapsed} мс)", deleted, stopwatch.ElapsedMilliseconds);
        return deleted;
    }
}
```

```
using System.Diagnostics;
using Microsoft.EntityFrameworkCore;
using Taxi.Data;
using Taxi.Logging;
using Taxi.Models;
```

```
namespace Taxi.Repositories;


public class DriverRepository : IDriverRepository

{

   private readonly TaxiDbContext context;

   private readonly ILogger<DriverRepository> logger;


   public DriverRepository(TaxiDbContext context, ILogger<DriverRepository>
logger)

   {

       this.context = context;

       this.logger = logger;

   }


   public async Task<List<Driver>> GetAllAsync()

   {

       var stopwatch = Stopwatch.StartNew();

       var drivers = await context.Drivers.AsNoTracking().OrderBy(d =>
d.Id).ToListAsync();

       logger.LogDebug(LogEvents.DbRead, "[БД] Отримано водіїв: {Count}
({Elapsed} мс)", drivers.Count, stopwatch.ElapsedMilliseconds);

       return drivers;

   }


   public async Task<Driver?> GetByIdAsync(int id)

   {

       var stopwatch = Stopwatch.StartNew();

       var driver = await context.Drivers.FirstOrDefaultAsync(d => d.Id == id);

       logger.LogDebug(LogEvents.DbRead, "[БД] Пошук водія ID {Id}: {Result}
({Elapsed} мс)", id, driver is null ? "не знайдено" : "знайдено",
stopwatch.ElapsedMilliseconds);

       return driver;

   }
```

```
    public async Task<List<Driver>> GetByCarIdAsync(int carId)
    {
        var stopwatch = Stopwatch.StartNew();
        var drivers = await context.Drivers.AsNoTracking().Where(d => d.CarId == carId).OrderBy(d => d.Id).ToListAsync();
        logger.LogDebug(LogEvents.DbRead, "[БД] Отримано водіїв автомобіля ID {CarId}: {Count} ({Elapsed} мс)", carId, drivers.Count, stopwatch.ElapsedMilliseconds);
        return drivers;
    }

    public async Task<int> CountByCarIdAsync(int carId)
    {
        var stopwatch = Stopwatch.StartNew();
        var count = await context.Drivers.CountAsync(d => d.CarId == carId);
        logger.LogDebug(LogEvents.DbRead, "[БД] Кількість водіїв автомобіля ID {CarId}: {Count} ({Elapsed} мс)", carId, count, stopwatch.ElapsedMilliseconds);
        return count;
    }

    public async Task AddAsync(Driver driver)
    {
        var stopwatch = Stopwatch.StartNew();
        await context.Drivers.AddAsync(driver);
        await context.SaveChangesAsync();
        logger.LogDebug(LogEvents.DbWrite, "[БД] Додано водія ID {Id} ({Elapsed} мс)", driver.Id, stopwatch.ElapsedMilliseconds);
    }

    public async Task UpdateAsync(Driver driver)
    {
        var stopwatch = Stopwatch.StartNew();
        context.Drivers.Update(driver);
        await context.SaveChangesAsync();
```

```
        logger.LogDebug(LogEvents.DbWrite, "[БД] Оновлено водія ID {Id}
({Elapsed} мс)", driver.Id, stopwatch.ElapsedMilliseconds);
    }

    public async Task DeleteAsync(Driver driver)
    {
        var stopwatch = Stopwatch.StartNew();
        context.Drivers.Remove(driver);
        await context.SaveChangesAsync();
        logger.LogDebug(LogEvents.DbDelete, "[БД] Видалено водія ID {Id}
({Elapsed} мс)", driver.Id, stopwatch.ElapsedMilliseconds);
    }

    public async Task<int> DeleteAllAsync()
    {
        var stopwatch = Stopwatch.StartNew();
        var deleted = await context.Drivers.ExecuteDeleteAsync();
        logger.LogDebug(LogEvents.DbDelete, "[БД] Видалено всіх водіїв: {Count}
({Elapsed} мс)", deleted, stopwatch.ElapsedMilliseconds);
        return deleted;
    }
}
```

```
using Taxi.Models;

namespace Taxi.Repositories;

public interface ICarRepository
{
    Task<List<Car>> GetAllAsync();
    Task<Car?> GetByIdAsync(int id);
    Task<bool> ExistsAsync(int id);
    Task AddAsync(Car car);
    Task UpdateAsync(Car car);
```

```
    Task DeleteAsync(Car car);

    Task<int> DeleteAllAsync();

}
```

```
using Taxi.Models;


namespace Taxi.Repositories;


public interface IDriverRepository

{

    Task<List<Driver>> GetAllAsync();

    Task<Driver?> GetByIdAsync(int id);

    Task<List<Driver>> GetByCarIdAsync(int carId);

    Task<int> CountByCarIdAsync(int carId);

    Task AddAsync(Driver driver);

    Task UpdateAsync(Driver driver);

    Task DeleteAsync(Driver driver);

    Task<int> DeleteAllAsync();

}
```

## Services

```
using AutoMapper;

using FluentValidation;

using Microsoft.Extensions.Options;

using Taxi.Caching;

using Taxi.DTOs;

using Taxi.Exceptions;

using Taxi.Logging;

using Taxi.Models;

using Taxi.Options;

using Taxi.Repositories;


namespace Taxi.Services;
```

```
public class CarService : ICarService
{
    private readonly ICarRepository carRepository;
    private readonly IDriverRepository driverRepository;
    private readonly IMapper mapper;
    private readonly IValidator<CarCreateDto> createValidator;
    private readonly IValidator<CarUpdateDto> updateValidator;
    private readonly ICacheService cache;
    private readonly IOptionsMonitor<TaxiFleetOptions> fleetOptions;
    private readonly ILogger<CarService> logger;

    public CarService(
        ICarRepository carRepository,
        IDriverRepository driverRepository,
        IMapper mapper,
        IValidator<CarCreateDto> createValidator,
        IValidator<CarUpdateDto> updateValidator,
        ICacheService cache,
        IOptionsMonitor<TaxiFleetOptions> fleetOptions,
        ILogger<CarService> logger)
    {
        this.carRepository = carRepository;
        this.driverRepository = driverRepository;
        this.mapper = mapper;
        this.createValidator = createValidator;
        this.updateValidator = updateValidator;
        this.cache = cache;
        this.fleetOptions = fleetOptions;
        this.logger = logger;
    }

    public async Task<List<CarResponseDto>> GetAllAsync()
```

```
    {
        logger.LogInformation(LogEvents.CarsGetAll, "Запит на отримання всіх
автомобілів");

        return await cache.GetOrCreateAsync(CacheKeys.AllCars, async () =>
        {
            var cars = await carRepository.GetAllAsync();
            return mapper.Map<List<CarResponseDto>>(cars);
        });
    }

    public async Task<CarResponseDto> GetByIdAsync(int id)
    {
        logger.LogInformation(LogEvents.CarGetById, "Запит на отримання
автомобіля ID {Id}", id);

        return await cache.GetOrCreateAsync(CacheKeys.Car(id), async () =>
        {
            var car = await GetExistingCarAsync(id);
            return mapper.Map<CarResponseDto>(car);
        });
    }

    public async Task<CarResponseDto> CreateAsync(CarCreateDto dto)
    {
        await createValidator.ValidateAndThrowAsync(dto);

        var car = mapper.Map<Car>(dto);
        await carRepository.AddAsync(car);

        cache.Remove(CacheKeys.AllCars);

        logger.LogInformation(LogEvents.CarCreated, "[Успіх] Додано автомобіль
{Brand} {Model}, ID {Id}",
```

```
            car.Brand, car.Model, car.Id);

        return mapper.Map<CarResponseDto>(car);
    }


    public async Task<CarResponseDto> UpdateAsync(int id, CarUpdateDto dto)
    {
        await updateValidator.ValidateAndThrowAsync(dto);


        var car = await GetExistingCarAsync(id);
        mapper.Map(dto, car);
        await carRepository.UpdateAsync(car);


        cache.Remove(CacheKeys.AllCars, CacheKeys.Car(id));


        logger.LogInformation(LogEvents.CarUpdated, "[Успіх] Оновлено автомобіль
ID {Id}", id);
        return mapper.Map<CarResponseDto>(car);
    }


    public async Task DeleteAsync(int id)
    {
        var car = await GetExistingCarAsync(id);
        await carRepository.DeleteAsync(car);


        cache.Clear();


        logger.LogInformation(LogEvents.CarDeleted, "[Успіх] Видалено автомобіль
ID {Id} разом з його водіями", id);
    }


    public async Task DeleteAllAsync()
    {
        if (!fleetOptions.CurrentValue.AllowDeleteAll)
        {
```

```
            logger.LogWarning(LogEvents.Forbidden, "[Заборонено] Спроба масового видалення автомобілів при TaxiFleet:AllowDeleteAll = false");
            throw new ForbiddenException("Масове видалення автомобілів заборонено налаштуваннями сервера.");
        }

        var deleted = await carRepository.DeleteAllAsync();
        cache.Clear();

        logger.LogWarning(LogEvents.CarsDeletedAll, "[Увага] Видалено всі автомобілі ({Count}) разом з їхніми водіями", deleted);
    }

    public async Task<List<DriverResponseDto>> GetDriversByCarIdAsync(int carId)
    {
        logger.LogInformation(LogEvents.CarDriversGet, "Запит на отримання водіїв автомобіля ID {CarId}", carId);

        return await cache.GetOrCreateAsync(CacheKeys.CarDrivers(carId), async () =>
        {
            if (!await carRepository.ExistsAsync(carId))
            {
                logger.LogWarning(LogEvents.CarNotFound, "Автомобіль ID {Id} не знайдено", carId);
                throw new NotFoundException($"Автомобіль з ID {carId} не знайдено.");
            }

            var drivers = await driverRepository.GetByCarIdAsync(carId);
            return mapper.Map<List<DriverResponseDto>>(drivers);
        });
    }

    private async Task<Car> GetExistingCarAsync(int id)
```

```
    {
        var car = await carRepository.GetByIdAsync(id);
        if (car is null)
        {
            logger.LogWarning(LogEvents.CarNotFound, "Автомобіль ID {Id} не
знайдено", id);
            throw new NotFoundException($"Автомобіль з ID {id} не знайдено.");
        }

        return car;
    }
}
```

```
using AutoMapper;
using FluentValidation;
using Microsoft.Extensions.Options;
using Taxi.Caching;
using Taxi.DTOs;
using Taxi.Exceptions;
using Taxi.Logging;
using Taxi.Models;
using Taxi.Options;
using Taxi.Repositories;

namespace Taxi.Services;

public class DriverService : IDriverService
{
    private readonly IDriverRepository driverRepository;
    private readonly ICarRepository carRepository;
    private readonly IMapper mapper;
    private readonly IValidator<DriverCreateDto> createValidator;
    private readonly IValidator<DriverUpdateDto> updateValidator;
```

```
    private readonly ICacheService cache;
    private readonly IOptionsMonitor<TaxiFleetOptions> fleetOptions;
    private readonly ILogger<DriverService> logger;

    public DriverService(
        IDriverRepository driverRepository,
        ICarRepository carRepository,
        IMapper mapper,
        IValidator<DriverCreateDto> createValidator,
        IValidator<DriverUpdateDto> updateValidator,
        ICacheService cache,
        IOptionsMonitor<TaxiFleetOptions> fleetOptions,
        ILogger<DriverService> logger)
    {
        this.driverRepository = driverRepository;
        this.carRepository = carRepository;
        this.mapper = mapper;
        this.createValidator = createValidator;
        this.updateValidator = updateValidator;
        this.cache = cache;
        this.fleetOptions = fleetOptions;
        this.logger = logger;
    }

    public async Task<List<DriverResponseDto>> GetAllAsync()
    {
        logger.LogInformation(LogEvents.DriversGetAll, "Запит на отримання всіх
водіїв");

        return await cache.GetOrCreateAsync(CacheKeys.AllDrivers, async () =>
        {
            var drivers = await driverRepository.GetAllAsync();
            return mapper.Map<List<DriverResponseDto>>(drivers);
```

```
        });
    }

    public async Task<DriverResponseDto> GetByIdAsync(int id)
    {
        logger.LogInformation(LogEvents.DriverGetById, "Запит на отримання водія
ID {Id}", id);

        return await cache.GetOrCreateAsync(CacheKeys.Driver(id), async () =>
        {
            var driver = await GetExistingDriverAsync(id);
            return mapper.Map<DriverResponseDto>(driver);
        });
    }

    public async Task<DriverResponseDto> CreateAsync(DriverCreateDto dto)
    {
        await createValidator.ValidateAndThrowAsync(dto);
        await EnsureCarExistsAsync(dto.CarId);
        await EnsureCarHasFreeSlotAsync(dto.CarId);

        var driver = mapper.Map<Driver>(dto);
        await driverRepository.AddAsync(driver);

        cache.Remove(CacheKeys.AllDrivers, CacheKeys.CarDrivers(driver.CarId));

        logger.LogInformation(LogEvents.DriverCreated, "[Успіх] Додано водія
{FullName}, ID {Id}", driver.FullName, driver.Id);
        return mapper.Map<DriverResponseDto>(driver);
    }

    public async Task<DriverResponseDto> UpdateAsync(int id, DriverUpdateDto
dto)
    {
```

```
        await updateValidator.ValidateAndThrowAsync(dto);

        var driver = await GetExistingDriverAsync(id);
        var oldCarId = driver.CarId;

        if (oldCarId != dto.CarId)
        {
            await EnsureCarExistsAsync(dto.CarId);
            await EnsureCarHasFreeSlotAsync(dto.CarId);
        }

        mapper.Map(dto, driver);
        await driverRepository.UpdateAsync(driver);

        cache.Remove(CacheKeys.AllDrivers, CacheKeys.Driver(id),
CacheKeys.CarDrivers(oldCarId), CacheKeys.CarDrivers(driver.CarId));

        logger.LogInformation(LogEvents.DriverUpdated, "[Успіх] Оновлено водія
ID {Id}", id);
        return mapper.Map<DriverResponseDto>(driver);
    }

    public async Task DeleteAsync(int id)
    {
        var driver = await GetExistingDriverAsync(id);
        var carId = driver.CarId;

        await driverRepository.DeleteAsync(driver);

        cache.Remove(CacheKeys.AllDrivers, CacheKeys.Driver(id),
CacheKeys.CarDrivers(carId));

        logger.LogInformation(LogEvents.DriverDeleted, "[Успіх] Видалено водія
ID {Id}", id);
```

```
    }

    public async Task DeleteAllAsync()
    {
        if (!fleetOptions.CurrentValue.AllowDeleteAll)
        {
            logger.LogWarning(LogEvents.Forbidden, "[Заборонено] Спроба масового видалення водіїв при TaxiFleet:AllowDeleteAll = false");
            throw new ForbiddenException("Масове видалення водіїв заборонено налаштуваннями сервера.");
        }

        var deleted = await driverRepository.DeleteAllAsync();
        cache.Clear();

        logger.LogWarning(LogEvents.DriversDeletedAll, "[Увага] Видалено всіх водіїв ({Count})", deleted);
    }

    private async Task<Driver> GetExistingDriverAsync(int id)
    {
        var driver = await driverRepository.GetByIdAsync(id);
        if (driver is null)
        {
            logger.LogWarning(LogEvents.DriverNotFound, "Водія ID {Id} не знайдено", id);
            throw new NotFoundException($"Водія з ID {id} не знайдено.");
        }

        return driver;
    }

    private async Task EnsureCarExistsAsync(int carId)
    {
```

```
        if (!await carRepository.ExistsAsync(carId))
        {
            logger.LogWarning(LogEvents.DriverCarMissing, "Спроба прив'язати водія до неіснуючого автомобіля ID {CarId}", carId);
            throw new BadRequestException($"Автомобіль з ID {carId} не існує.");
        }
    }

    private async Task EnsureCarHasFreeSlotAsync(int carId)
    {
        var maxDrivers = fleetOptions.CurrentValue.MaxDriversPerCar;
        var currentCount = await driverRepository.CountByCarIdAsync(carId);

        if (currentCount >= maxDrivers)
        {
            logger.LogWarning(LogEvents.DriverLimitReached, "[Ліміт] За автомобілем ID {CarId} вже {Count} водіїв (максимум {Max})", carId, currentCount, maxDrivers);
            throw new ConflictException($"За автомобілем з ID {carId} вже закріплено максимальну кількість водіїв ({maxDrivers}).");
        }
    }
}
```

```
using Taxi.DTOs;

namespace Taxi.Services;

public interface ICarService
{
    Task<List<CarResponseDto>> GetAllAsync();
    Task<CarResponseDto> GetByIdAsync(int id);
    Task<CarResponseDto> CreateAsync(CarCreateDto dto);
    Task<CarResponseDto> UpdateAsync(int id, CarUpdateDto dto);
```

```
    Task DeleteAsync(int id);

    Task DeleteAllAsync();

    Task<List<DriverResponseDto>> GetDriversByCarIdAsync(int carId);
}
```

```
using Taxi.DTOs;


namespace Taxi.Services;


public interface IDriverService
{
    Task<List<DriverResponseDto>> GetAllAsync();

    Task<DriverResponseDto> GetByIdAsync(int id);

    Task<DriverResponseDto> CreateAsync(DriverCreateDto dto);

    Task<DriverResponseDto> UpdateAsync(int id, DriverUpdateDto dto);

    Task DeleteAsync(int id);

    Task DeleteAllAsync();
}
```

## Validators

```
using FluentValidation;

using Microsoft.Extensions.Options;

using Taxi.DTOs;

using Taxi.Options;


namespace Taxi.Validators;


public abstract class CarBaseDtoValidator<T> : AbstractValidator<T> where T :
CarBaseDto

{

    protected CarBaseDtoValidator(IOptionsMonitor<TaxiFleetOptions>
fleetOptions)

    {
```

```
        RuleLevelCascadeMode = CascadeMode.Stop;


        var currentYear = DateTime.UtcNow.Year;
        var maxCarAge = fleetOptions.CurrentValue.MaxCarAgeYears;
        var minYear = currentYear - maxCarAge;
        var maxYear = currentYear + 1;


        RuleFor(x => x.Brand)
            .NotEmpty().WithMessage("Марка автомобіля є обов'язковою.")
            .MaximumLength(50).WithMessage("Марка автомобіля не може
перевищувати 50 символів.");


        RuleFor(x => x.Model)
            .NotEmpty().WithMessage("Модель автомобіля є обов'язковою.")
            .MaximumLength(50).WithMessage("Модель автомобіля не може
перевищувати 50 символів.");


        RuleFor(x => x.EngineVolume)
            .InclusiveBetween(0.0, 10.0).WithMessage("Об'єм двигуна має бути від
0 до 10 л (0 — для електромобілів).");


        RuleFor(x => x.Year)
            .InclusiveBetween(minYear, maxYear)
            .WithMessage($"Рік випуску має бути від {minYear} до {maxYear}: до
роботи в таксопарку допускаються автомобілі не старші за {maxCarAge} років.");
    }
}
```

```
using Microsoft.Extensions.Options;
using Taxi.DTOs;
using Taxi.Options;


namespace Taxi.Validators;
```

```
public class CarCreateDtoValidator : CarBaseDtoValidator<CarCreateDto>

{

   public CarCreateDtoValidator(IOptionsMonitor<TaxiFleetOptions> fleetOptions)
: base(fleetOptions)

   {

   }

}
```

```
using Microsoft.Extensions.Options;

using Taxi.DTOs;

using Taxi.Options;


namespace Taxi.Validators;


public class CarUpdateDtoValidator : CarBaseDtoValidator<CarUpdateDto>

{

   public CarUpdateDtoValidator(IOptionsMonitor<TaxiFleetOptions> fleetOptions)
: base(fleetOptions)

   {

   }

}
```

```
using FluentValidation;

using Taxi.DTOs;


namespace Taxi.Validators;


public abstract class DriverBaseDtoValidator<T> : AbstractValidator<T> where T
: DriverBaseDto

{

   protected DriverBaseDtoValidator()

   {

       RuleLevelCascadeMode = CascadeMode.Stop;
```

```
        RuleFor(x => x.FullName)
            .NotEmpty().WithMessage("ПІБ водія є обов'язковим.")
            .MinimumLength(3).WithMessage("ПІБ водія має містити щонайменше 3 символи.")
            .MaximumLength(100).WithMessage("ПІБ водія не може перевищувати 100 символів.")
            .Must(ValidationRules.IsValidPersonName).WithMessage("ПІБ може містити лише літери, пробіли, дефіс та апостроф.");

        RuleFor(x => x.Rating)
            .InclusiveBetween(0.0, 5.0).WithMessage("Рейтинг водія має бути в межах від 0 до 5.");

        RuleFor(x => x.CarId)
            .GreaterThan(0).WithMessage("Ідентифікатор автомобіля (CarId) має бути додатним числом.");
    }
}
```

```
using Taxi.DTOs;

namespace Taxi.Validators;

public class DriverCreateDtoValidator : DriverBaseDtoValidator<DriverCreateDto>
{
}
```

```
using Taxi.DTOs;

namespace Taxi.Validators;

public class DriverUpdateDtoValidator : DriverBaseDtoValidator<DriverUpdateDto>
{
}
```

```
using System.Text.RegularExpressions;


namespace Taxi.Validators;


public static class ValidationRules

{

   private static readonly Regex PersonNameRegex =

       new(@"^\p{L}+(?:[ '’\-]\p{L}+)*$", RegexOptions.Compiled);


   public static bool IsValidPersonName(string? value) =>

       !string.IsNullOrWhiteSpace(value) &&
PersonNameRegex.IsMatch(value.Trim());

}
```

## Program

```
using System.Text.Encodings.Web;

using System.Text.Unicode;

using AutoMapper;

using FluentValidation;

using Microsoft.EntityFrameworkCore;

using Serilog;

using Taxi.Caching;

using Taxi.Data;

using Taxi.Mappings;

using Taxi.Middleware;

using Taxi.Options;

using Taxi.Repositories;

using Taxi.Services;

using Taxi.Validators;


var builder = WebApplication.CreateBuilder(args);

builder.Logging.ClearProviders();
```

```
builder.Host.UseSerilog((context, config) =>
config.ReadFrom.Configuration(context.Configuration));


builder.Services
    .AddControllers(mvcOptions =>
    {
        mvcOptions.SuppressImplicitRequiredAttributeForNonNullableReferenceTypes
= true;
    })
    .AddJsonOptions(jsonOptions =>
    {
        jsonOptions.JsonSerializerOptions.Encoder =
JavaScriptEncoder.Create(UnicodeRanges.All);
    });


builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();


builder.Services.AddOptions<TaxiFleetOptions>().BindConfiguration(TaxiFleetOpti
ons.SectionName).ValidateDataAnnotations().ValidateOnStart();


builder.Services.AddOptions<CachingOptions>().BindConfiguration(CachingOptions.
SectionName).ValidateDataAnnotations().ValidateOnStart();


var connectionString =
builder.Configuration.GetConnectionString("DefaultConnection") ?? throw new
InvalidOperationException("Рядок підключення 'DefaultConnection' не
знайдено.");


builder.Services.AddDbContext<TaxiDbContext>(dbOptions =>
dbOptions.UseNpgsql(connectionString));


builder.Services.AddMemoryCache();
builder.Services.AddSingleton<ICacheService, CacheService>();


builder.Services.AddScoped<ICarRepository, CarRepository>();
```

```
builder.Services.AddScoped<IDriverRepository, DriverRepository>();


builder.Services.AddScoped<ICarService, CarService>();
builder.Services.AddScoped<IDriverService, DriverService>();


builder.Services.AddValidatorsFromAssemblyContaining<CarCreateDtoValidator>();
builder.Services.AddAutoMapper(typeof(MappingProfile));


var app = builder.Build();


app.UseSerilogRequestLogging();


app.UseMiddleware<ExceptionHandlingMiddleware>();


if (app.Environment.IsDevelopment())
{
   app.UseSwagger();
   app.UseSwaggerUI();



app.Services.GetRequiredService<IMapper>().ConfigurationProvider.AssertConfigur
ationIsValid();


   await using var scope = app.Services.CreateAsyncScope();
   var db = scope.ServiceProvider.GetRequiredService<TaxiDbContext>();


   try
   {
       if (db.Database.GetMigrations().Any())
       {
           await db.Database.MigrateAsync();
       }
   }
   catch (Exception ex)
```

```
    {
        app.Logger.LogError(ex, "Не вдалося підключитися до PostgreSQL або застосувати міграції.");
    }
}


app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();


await app.RunAsync();
```

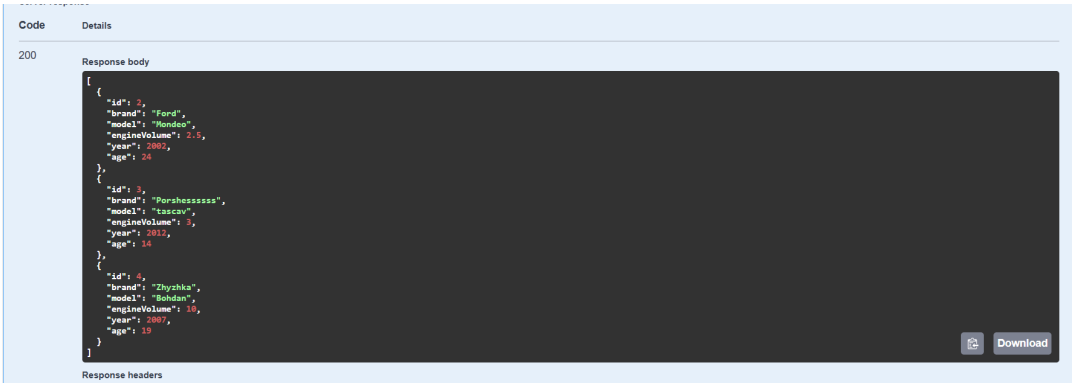
## appsettings.json

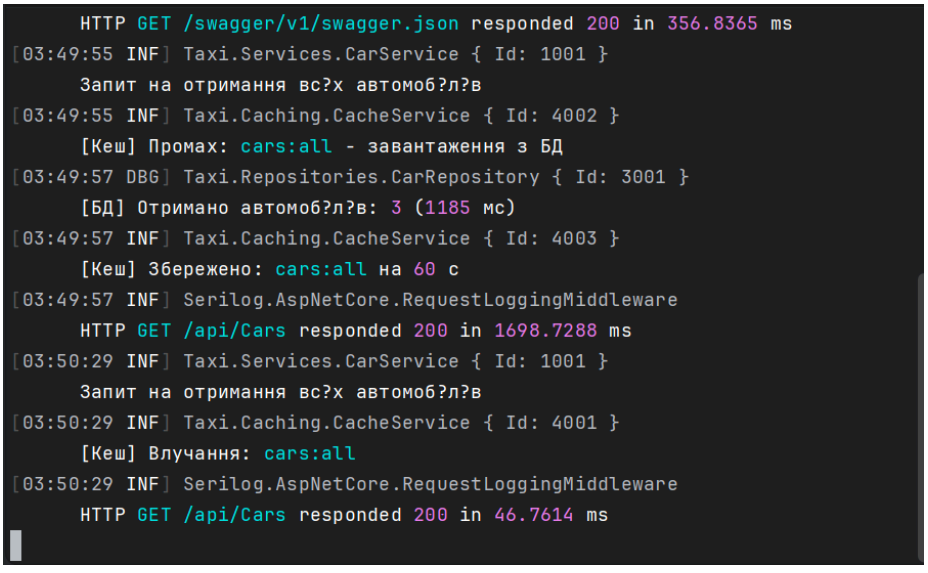
```
{
 "ConnectionStrings": {
   "DefaultConnection": "Host=localhost;Port=5432;Database=taxi_lab2;Username=postgres;Password=your_password"
 },
 "Serilog": {
   "Using": [ "Serilog.Sinks.Console", "Serilog.Sinks.File", "Serilog.Enrichers.Thread", "Serilog.Enrichers.Environment" ],
   "MinimumLevel": {
     "Default": "Information",
     "Override": {
       "Microsoft": "Warning",
       "Microsoft.AspNetCore.Hosting": "Warning",
       "Microsoft.AspNetCore.Mvc": "Warning",
       "Microsoft.AspNetCore.Routing": "Warning",
       "Microsoft.EntityFrameworkCore": "Warning",
       "Microsoft.Hosting.Lifetime": "Information",
       "Taxi.Repositories": "Debug"
     }
   },
```

```
  "WriteTo": [
    {
      "Name": "Console",
      "Args": {
        "outputTemplate": "[{Timestamp:HH:mm:ss} {Level:u3}] {SourceContext}
{EventId}{NewLine}      {Message:lj}{NewLine}{Exception}"
      }
    },
    {
      "Name": "File",
      "Args": {
        "path": "logs/taxi-.log",
        "rollingInterval": "Day",
        "outputTemplate": "{Timestamp:yyyy-MM-dd HH:mm:ss.fff} [{Level:u3}]
[Thread {ThreadId}] {SourceContext} {EventId}:
{Message:lj}{NewLine}{Exception}"
      }
    },
    {
      "Name": "File",
      "Args": {
        "path": "logs/taxi-.json",
        "rollingInterval": "Day",
        "formatter": "Serilog.Formatting.Compact.CompactJsonFormatter,
Serilog.Formatting.Compact"
      }
    }
  ],
  "Enrich": [ "FromLogContext", "WithThreadId", "WithMachineName" ]
 },
 "TaxiFleet": {
  "MaxDriversPerCar": 3,
  "MaxCarAgeYears": 15,
  "AllowDeleteAll": false
```

```
  },
  "Caching": {
    "Enabled": true,
    "DurationSeconds": 60
  },
  "AllowedHosts": "*"
}
```

## Знімки екрану з прикладами результатів роботи розробленої програми






```
      [БД] Додано автомоб?ль ID 5 (1126 мс)
[03:52:07 INF] Taxi.Caching.CacheService { Id: 4004 }
      [Кеш] Скинуто: cars:all
[03:52:07 INF] Taxi.Services.CarService { Id: 1003 }
      [Усп?х] Додано автомоб?ль Renaut Megan, ID 5
[03:52:08 INF] Serilog.AspNetCore.RequestLoggingMiddleware
      HTTP POST /api/Cars responded 201 in 1504.0448 ms
[03:52:32 INF] Taxi.Services.CarService { Id: 1001 }
      Запит на отримання вс?х автомоб?л?в
[03:52:32 INF] Taxi.Caching.CacheService { Id: 4002 }
      [Кеш] Промах: cars:all - завантаження з БД
[03:52:32 DBG] Taxi.Repositories.CarRepository { Id: 3001 }
      [БД] Отримано автомоб?л?в: 4 (13 мс)
[03:52:32 INF] Taxi.Caching.CacheService { Id: 4003 }
      [Кеш] Збережено: cars:all на 60 с
[03:52:32 INF] Serilog.AspNetCore.RequestLoggingMiddleware
      HTTP GET /api/Cars responded 200 in 99.6360 ms
[03:52:33 INF] Taxi.Services.CarService { Id: 1001 }
      Запит на отримання вс?х автомоб?л?в
[03:52:33 INF] Taxi.Caching.CacheService { Id: 4001 }
      [Кеш] Влучання: cars:all
[03:52:33 INF] Serilog.AspNetCore.RequestLoggingMiddleware
      HTTP GET /api/Cars responded 200 in 37.6925 ms
```

**Code** **Details**

400 Error: response status is 400

Response body

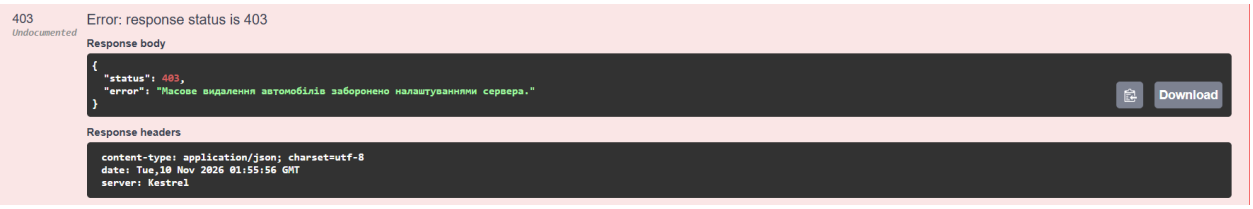
```
{
  "status": 400,
  "error": "Помилка валідації вхідних даних.",
  "errors": {
    "Brand": [
      "Марка автомобіля є обов'язковою."
    ],
    "Year": [
      "Рік випуску має бути від 2011 до 2027: до роботи в таксопарку допускаються автомобілі не старші за 15 років."
    ]
  }
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Tue,10 Nov 2026 01:53:32 GMT
server: Kestrel
```

```
2026-11-10 03:49:48.882 [INF] [Thread 8] Microsoft.Hosting.Lifetime { Id: 14, Name: "ListeningOnAddress" }: Now listeni
2026-11-10 03:49:49.142 [INF] [Thread 8] Microsoft.Hosting.Lifetime { Id: 14, Name: "ListeningOnAddress" }: Now listening on: h
2026-11-10 03:49:49.159 [INF] [Thread 8] Microsoft.Hosting.Lifetime : Application started. Press Ctrl+C to shut down.
2026-11-10 03:49:49.167 [INF] [Thread 8] Microsoft.Hosting.Lifetime : Hosting environment: Development
2026-11-10 03:49:49.178 [INF] [Thread 8] Microsoft.Hosting.Lifetime : Content root path: D:\Desktop\Lab_Web\TaxiLab3\Taxi
2026-11-10 03:49:50.000 [INF] [Thread 10] Serilog.AspNetCore.RequestLoggingMiddleware : HTTP GET /swagger/index.html responded
2026-11-10 03:49:51.685 [INF] [Thread 10] Serilog.AspNetCore.RequestLoggingMiddleware : HTTP GET /swagger/v1/swagger.json respo
2026-11-10 03:49:55.785 [INF] [Thread 5] Taxi.Services.CarService { Id: 1001 }: Запит на отримання всіх автомобілів
2026-11-10 03:49:55.809 [INF] [Thread 5] Taxi.Caching.CacheService { Id: 4002 }: [Кеш] Промах: cars:all — завантаження з БД
2026-11-10 03:49:57.032 [DBG] [Thread 10] Taxi.Repositories.CarRepository { Id: 3001 }: [БД] Отримано автомобілів: 3 (1185 мс)
2026-11-10 03:49:57.120 [INF] [Thread 10] Taxi.Caching.CacheService { Id: 4003 }: [Кеш] Збережено: cars:all на 60 с
2026-11-10 03:49:57.307 [INF] [Thread 10] Serilog.AspNetCore.RequestLoggingMiddleware : HTTP GET /api/Cars responded 200 in 169
2026-11-10 03:50:29.889 [INF] [Thread 10] Taxi.Services.CarService { Id: 1001 }: Запит на отримання всіх автомобілів
2026-11-10 03:50:29.905 [INF] [Thread 10] Taxi.Caching.CacheService { Id: 4001 }: [Кеш] Влучання: cars:all
2026-11-10 03:50:29.926 [INF] [Thread 10] Serilog.AspNetCore.RequestLoggingMiddleware : HTTP GET /api/Cars responded 200 in 46.
2026-11-10 03:52:07.929 [DBG] [Thread 13] Taxi.Repositories.CarRepository { Id: 3002 }: [БД] Додано автомобіль ID 5 (1126 мс)
2026-11-10 03:52:07.952 [INF] [Thread 13] Taxi.Caching.CacheService { Id: 4004 }: [Кеш] Скинуто: cars:all
2026-11-10 03:52:07.967 [INF] [Thread 13] Taxi.Services.CarService { Id: 1003 }: [Успіх] Додано автомобіль Renaut Megan, ID 5
2026-11-10 03:52:08.078 [INF] [Thread 13] Serilog.AspNetCore.RequestLoggingMiddleware : HTTP POST /api/Cars responded 201 in 15
2026-11-10 03:52:32.709 [INF] [Thread 13] Taxi.Services.CarService { Id: 1001 }: Запит на отримання всіх автомобілів
2026-11-10 03:52:32.736 [INF] [Thread 13] Taxi.Caching.CacheService { Id: 4002 }: [Кеш] Промах: cars:all — завантаження з БД
2026-11-10 03:52:32.768 [DBG] [Thread 8] Taxi.Repositories.CarRepository { Id: 3001 }: [БД] Отримано автомобілів: 4 (13 мс)
2026-11-10 03:52:32.786 [INF] [Thread 8] Taxi.Caching.CacheService { Id: 4003 }: [Кеш] Збережено: cars:all на 60 с
2026-11-10 03:52:32.802 [INF] [Thread 8] Serilog.AspNetCore.RequestLoggingMiddleware : HTTP GET /api/Cars responded 200 in 99.6
2026-11-10 03:52:33.465 [INF] [Thread 3] Taxi.Services.CarService { Id: 1001 }: Запит на отримання всіх автомобілів
2026-11-10 03:52:33.484 [INF] [Thread 3] Taxi.Caching.CacheService { Id: 4001 }: [Кеш] Влучання: cars:all
2026-11-10 03:52:33.501 [INF] [Thread 3] Serilog.AspNetCore.RequestLoggingMiddleware : HTTP GET /api/Cars responded 200 in 37.6
2026-11-10 03:53:33.452 [WRN] [Thread 13] Taxi.Middleware.ExceptionHandlingMiddleware { Id: 5400 }: [Валідація] POST /api/Cars:
2026-11-10 03:53:33.482 [INF] [Thread 13] Serilog.AspNetCore.RequestLoggingMiddleware : HTTP POST /api/Cars responded 400 in 97
```




```
[03:52:33 INF] Taxi.Caching.CacheService { Id: 4001 }
      [Кеш] Влучання: cars:all
[03:52:33 INF] Serilog.AspNetCore.RequestLoggingMiddleware
      HTTP GET /api/Cars responded 200 in 37.6925 ms
[03:53:33 WRN] Taxi.Middleware.ExceptionHandlingMiddleware { Id: 5400 }
      [Вал?дац?я] POST /api/Cars: Brand - Марка автомоб?ля є обов'язковою.; Ye
ar - Р?к випуску має бути в?д 2011 до 2027: до роботи в таксопарку допускаютьс
я автомоб?л? не старш? за 15 рок?в.
[03:53:33 INF] Serilog.AspNetCore.RequestLoggingMiddleware
      HTTP POST /api/Cars responded 400 in 97.7677 ms
[03:55:57 WRN] Taxi.Services.CarService { Id: 5403 }
      [Заборонено] Спроба масового видалення автомоб?л?в при TaxiFleet:AllowDe
leteAll = false
[03:55:57 INF] Serilog.AspNetCore.RequestLoggingMiddleware
      HTTP DELETE /api/Cars responded 403 in 34.5748 ms
```

## Висновок

Під час виконання комп’ютерного практикуму до REST API системи управління таксопарком було додано логування, кешування та роботу з конфігурацією. Для логування я використав бібліотеку Serilog із записом у консоль та файли. Кешування GET-запитів у пам’яті зменшило кількість звернень до бази даних. Бізнес-налаштування таксопарку винесено у файл конфігурації, і вони впливають на поведінку API. Усі необхідні об’єкти отримуються через IoC-контейнер.